

Лабораторная работа (Вариант 1, версия 2026)

Python + PyQt6 + MySQL (VS Code)

Автор: Каламбет В.Б.

подготовка к демонстрационному экзамену

Содержание

Шаг 1. Создайте рабочую структуру проекта	4
Шаг 2. Поднимите MySQL + phpMyAdmin через Docker Compose	4
Шаг 3. Создайте схему БД (модули 1–4)	5
Шаг 4. Подготовьте CSV из исходных XLSX (Через Excel)	9
Шаг 5. Импортируйте данные через UI phpMyAdmin	9
Шаг 5.1. Перенесите данные из raw в боевые таблицы	10
Шаг 6. Выполните контрольные SQL-проверки	12
Шаг 7. Создайте Python-проект и установите зависимости	13
Шаг 8. Создайте db.py	13
Шаг 8.1. Создайте check_db.py и проверьте подключение	14
Шаг 9. Создайте main.py (финальная версия модулей 2–4)	15
Шаг 10. Запустите приложение и проверьте роли	42
Шаг 11. Проверьте модули 2-4	42
Шаг 11.1. Зафиксируйте локальный git-коммит (модули 2-4)	43
Шаг 12. Подготовьте блок-схему алгоритма по ГОСТ и сохраните PDF	43
Шаг 13. Подготовьте DOCX со скриншотами корректной работы	44
Шаг 14. Экспортируйте SQL-скрипт БД и ER-диаграмму	44
Шаг 15. Соберите .exe через PyInstaller	45
Шаг 16. Подготовьте финальный набор файлов и git-коммит	45

Результат лабораторной

После выполнения у вас будет готовое приложение по модулям 1–4:

- база данных MySQL в 3НФ;
 - импорт исходных данных из `xlsx` (через `csv`) в `phpMyAdmin`;
 - окно входа и роли (гость, клиент, менеджер, администратор);
 - список товаров с подсветкой скидок/остатков, поиском, фильтром и сортировкой по ролям;
 - форма добавления/редактирования товара для администратора;
 - окно заказов и форма добавления/редактирования заказа;
 - ER-диаграмма БД в PDF, SQL-скрипт БД, `algorithm_gost.pdf`, `report_screenshots.docx`;
 - исполняемый `.exe` через `pyinstaller`;
 - локальный `git`-репозиторий с зафиксированной итоговой версией проекта.
-

Шаг 1. Создайте рабочую структуру проекта

Что делаем

Создайте папку проекта и подкаталоги для кода и ресурсов.

Команды/код

```
cd C:\
mkdir shoe_store_2026_pu_python
cd shoe_store_2026_pu_python
mkdir app
mkdir resources
mkdir resources\photos
mkdir docs
mkdir sql
mkdir screenshots
```

Скопируйте из папки

Вариант1(2026)\Данные\Модуль 1\Прил_2_03_КОД 09.02.07-2-2026-M1\import в C:\shoe_store_2026_pu_python\resources:

- Icon.ico
- Icon.png
- picture.png
- Товар.xlsx
- user_import.xlsx
- Пункты выдачи_import.xlsx
- Заказ_import.xlsx

Скопируйте папку photos в:

- C:\shoe_store_2026_pu_python\resources\photos
-

Шаг 2. Поднимите MySQL + phpMyAdmin через Docker Compose

Что делаем

Создайте docker-compose.yml и запустите контейнеры.

Альтернативно (без Docker) можно использовать XAMPP, Open Server Panel или локально установленный MySQL Server.

Команды/код

Создайте файл C:\shoe_store_2026_pu_python\docker-compose.yml:

```
services:
  mysql:
    image: mysql:8.4
    container_name: shoe2026_pu_mysql
```

```

restart: unless-stopped
environment:
  MYSQL_ROOT_PASSWORD: root
  MYSQL_DATABASE: shoe2026_pu
  MYSQL_USER: demo
  MYSQL_PASSWORD: demo
ports:
  - "3306:3306"
volumes:
  - mysql_data:/var/lib/mysql

phpmyadmin:
  image: phpmyadmin:latest
  container_name: shoe2026_pu_phpmyadmin
  restart: unless-stopped
  environment:
    PMA_HOST: mysql
    PMA_PORT: 3306
    PMA_USER: root
    PMA_PASSWORD: root
  ports:
    - "8081:80"
  depends_on:
    - mysql

volumes:
  mysql_data:

```

Запуск:

```

cd C:\shoe_store_2026_pu_python
docker compose up -d
docker compose ps

```

Откройте phpMyAdmin:

- при Docker: <http://localhost:8081>, логин root, пароль root;
- при XAMPP/Open Server Panel/локальном MySQL Server: ваш локальный адрес phpMyAdmin (часто <http://localhost/phpmyadmin>).

Шаг 3. Создайте схему БД (модули 1–4)

Что делаем

Создайте рабочие таблицы, связи и raw-таблицы для импорта.

Команды/код

1. Если БД shoe2026_pu еще не создана: вкладка **Базы данных** -> имя shoe2026_pu -> сравнение utf8mb4_unicode_ci -> **Создать**.

2. Выберите БД shoe2026_pu.

3. Откройте вкладку **SQL** и выполните:

```
USE shoe2026_pu;
```

```
SET NAMES utf8mb4;
```

```
CREATE TABLE roles (  
    role_id TINYINT UNSIGNED PRIMARY KEY,  
    role_name VARCHAR(60) NOT NULL UNIQUE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE users (  
    user_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    role_id TINYINT UNSIGNED NOT NULL,  
    full_name VARCHAR(200) NOT NULL,  
    login VARCHAR(120) NOT NULL UNIQUE,  
    password_plain VARCHAR(120) NOT NULL,  
    CONSTRAINT fk_users_roles  
        FOREIGN KEY (role_id) REFERENCES roles(role_id)  
        ON UPDATE CASCADE ON DELETE RESTRICT  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE categories (  
    category_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(120) NOT NULL UNIQUE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE manufacturers (  
    manufacturer_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(120) NOT NULL UNIQUE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE suppliers (  
    supplier_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(120) NOT NULL UNIQUE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
CREATE TABLE products (  
    product_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    article VARCHAR(50) NOT NULL UNIQUE,  
    name VARCHAR(200) NOT NULL,  
    unit_name VARCHAR(20) NOT NULL,  
    price DECIMAL(12,2) NOT NULL CHECK (price >= 0),  
    supplier_id INT UNSIGNED NOT NULL,  
    manufacturer_id INT UNSIGNED NOT NULL,  
    category_id INT UNSIGNED NOT NULL,  
    discount_percent DECIMAL(5,2) NOT NULL CHECK (discount_percent >= 0),  
    stock_quantity INT NOT NULL CHECK (stock_quantity >= 0),
```

```

description_text TEXT NULL,
photo_file VARCHAR(255) NULL,
CONSTRAINT fk_products_suppliers
    FOREIGN KEY (supplier_id) REFERENCES suppliers(supplier_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT fk_products_manufacturers
    FOREIGN KEY (manufacturer_id) REFERENCES manufacturers(manufacturer_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT fk_products_categories
    FOREIGN KEY (category_id) REFERENCES categories(category_id)
    ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE pickup_points (
    pickup_point_id INT UNSIGNED PRIMARY KEY,
    address_text VARCHAR(255) NOT NULL UNIQUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE order_statuses (
    status_id TINYINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    status_name VARCHAR(60) NOT NULL UNIQUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE orders (
    order_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    order_number INT UNSIGNED NOT NULL UNIQUE,
    article_text VARCHAR(255) NOT NULL,
    order_date DATE NULL,
    delivery_date DATE NULL,
    pickup_point_id INT UNSIGNED NOT NULL,
    client_user_id INT UNSIGNED NULL,
    pickup_code INT UNSIGNED NOT NULL,
    status_id TINYINT UNSIGNED NOT NULL,
    CONSTRAINT fk_orders_pickup_points
        FOREIGN KEY (pickup_point_id) REFERENCES pickup_points(pickup_point_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_orders_users
        FOREIGN KEY (client_user_id) REFERENCES users(user_id)
        ON UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT fk_orders_statuses
        FOREIGN KEY (status_id) REFERENCES order_statuses(status_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE order_items (
    order_item_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    order_id INT UNSIGNED NOT NULL,
    product_id INT UNSIGNED NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0),

```

```

CONSTRAINT fk_order_items_order
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
CONSTRAINT fk_order_items_product
    FOREIGN KEY (product_id) REFERENCES products(product_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
CONSTRAINT uk_order_product UNIQUE (order_id, product_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE INDEX idx_products_supplier ON products(supplier_id);
CREATE INDEX idx_products_category ON products(category_id);
CREATE INDEX idx_orders_status ON orders(status_id);
CREATE INDEX idx_order_items_product ON order_items(product_id);

CREATE TABLE users_import_raw (
    role_name VARCHAR(100),
    full_name VARCHAR(200),
    login_text VARCHAR(120),
    password_text VARCHAR(120)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE products_import_raw (
    article_text VARCHAR(60),
    name_text VARCHAR(200),
    unit_text VARCHAR(20),
    price_text VARCHAR(40),
    supplier_text VARCHAR(120),
    manufacturer_text VARCHAR(120),
    category_text VARCHAR(120),
    discount_text VARCHAR(40),
    stock_text VARCHAR(40),
    description_text TEXT,
    photo_text VARCHAR(120)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE pickup_points_import_raw (
    raw_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    address_text VARCHAR(255)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE orders_import_raw (
    raw_id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    order_number_text VARCHAR(40),
    articles_text VARCHAR(255),
    order_date_text VARCHAR(40),
    delivery_date_text VARCHAR(40),
    pickup_point_text VARCHAR(40),
    client_fio_text VARCHAR(200),
    pickup_code_text VARCHAR(40),

```



```
status_text VARCHAR(80)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Шаг 4. Подготовьте CSV из исходных XLSX (Через Excel)

Что делаем

Откройте каждый `xlsx` в Excel, подготовьте строки и сохраните CSV UTF-8.

Команды/код

Сначала подготовьте `xlsx`, затем сохраняйте `csv`:

1. В `user_import.xlsx`, `Tovar.xlsx`, `Заказ_import.xlsx` удалите первую строку с заголовками.
 2. В `Пункты выдачи_import.xlsx` первую строку не удаляйте (это данные).
 3. В `Заказ_import.xlsx` удалите пустые столбцы I, J, K, L.
 4. Сохраните как CSV UTF-8:
 - `user_import.csv`
 - `Tovar.csv`
 - `Пункты выдачи_import.csv`
 - `Заказ_import.csv`
 5. Откройте каждый `csv` в Блокноте и удалите пустые строки в конце файла (если есть).
 6. Проверьте символ-разделитель столбцов в файле (; или ,), этот символ используйте на шаге импорта.
-

Шаг 5. Импортируйте данные через UI phpMyAdmin

Что делаем

Импортируйте CSV в `raw`-таблицы.

Команды/код

В phpMyAdmin:

1. Таблица `users_import_raw` -> **Импорт** -> `user_import.csv` -> Названия столбцов: `role_name, full_name, login_text, password_text`
2. Таблица `products_import_raw` -> **Импорт** -> `Tovar.csv` -> Названия столбцов: `article_text, name_text, unit_text, price_text, supplier_text, manufacturer_text, category_text, discount_text`
3. Таблица `pickup_points_import_raw` -> **Импорт** -> `Пункты выдачи_import.csv` -> Названия столбцов: `address_text`
4. Таблица `orders_import_raw` -> **Импорт** -> `Заказ_import.csv` -> Названия столбцов: `order_number_text, articles_text, order_date_text, delivery_date_text, pickup_point_text, client_fio`

Для каждого импорта выставьте:

- Формат = CSV
 - Кодировка файла = utf-8
 - Разделитель полей = символ вашего CSV (; или ,)
 - Значения полей обрамлены = "
 - Символ экранирования = \"
 - Разделитель строк = auto
 - Названия столбцов = как указано выше
-

Шаг 5.1. Перенесите данные из raw в боевые таблицы

Что делаем

Заполните рабочие таблицы и связи.

Команды/код

Во вкладке **SQL** выполните:

```
INSERT INTO roles (role_id, role_name) VALUES
(1, 'Гость'),
(2, 'Авторизованный клиент'),
(3, 'Менеджер'),
(4, 'Администратор');

INSERT INTO users (role_id, full_name, login, password_plain)
SELECT
CASE
WHEN role_name LIKE '%Администратор%' THEN 4
WHEN role_name LIKE '%Менеджер%' THEN 3
ELSE 2
END AS role_id,
TRIM(full_name),
TRIM(login_text),
TRIM(REPLACE(password_text, '\r', ''))
FROM users_import_raw
WHERE TRIM(login_text) <> '';

INSERT INTO categories (name)
SELECT DISTINCT TRIM(category_text)
FROM products_import_raw
WHERE TRIM(category_text) <> '';

INSERT INTO manufacturers (name)
SELECT DISTINCT TRIM(manufacturer_text)
FROM products_import_raw
WHERE TRIM(manufacturer_text) <> '';

INSERT INTO suppliers (name)
SELECT DISTINCT TRIM(supplier_text)
```

```

FROM products_import_raw
WHERE TRIM(supplier_text) <> '';

INSERT INTO products (
    article, name, unit_name, price,
    supplier_id, manufacturer_id, category_id,
    discount_percent, stock_quantity, description_text, photo_file
)
SELECT
    TRIM(p.article_text),
    TRIM(p.name_text),
    TRIM(p.unit_text),
    CAST(REPLACE(TRIM(p.price_text), ',', '.') AS DECIMAL(12,2)),
    s.supplier_id,
    m.manufacturer_id,
    c.category_id,
    CAST(REPLACE(TRIM(p.discount_text), ',', '.') AS DECIMAL(5,2)),
    CAST(TRIM(p.stock_text) AS UNSIGNED),
    NULLIF(TRIM(p.description_text), ''),
    NULLIF(TRIM(REPLACE(p.photo_text, '\r', '')), '')
FROM products_import_raw p
JOIN suppliers s ON s.name = TRIM(p.supplier_text)
JOIN manufacturers m ON m.name = TRIM(p.manufacturer_text)
JOIN categories c ON c.name = TRIM(p.category_text)
WHERE TRIM(p.article_text) <> '';

INSERT INTO pickup_points (pickup_point_id, address_text)
SELECT raw_id, TRIM(REPLACE(address_text, '\r', ''))
FROM pickup_points_import_raw
WHERE TRIM(REPLACE(address_text, '\r', '')) <> ''
ORDER BY raw_id;

INSERT INTO order_statuses (status_name)
SELECT DISTINCT TRIM(REPLACE(status_text, '\r', ''))
FROM orders_import_raw
WHERE TRIM(REPLACE(status_text, '\r', '')) <> '';

INSERT INTO orders (
    order_number, article_text, order_date, delivery_date,
    pickup_point_id, client_user_id, pickup_code, status_id
)
SELECT
    CAST(TRIM(o.order_number_text) AS UNSIGNED),
    TRIM(o.articles_text),
    STR_TO_DATE(TRIM(o.order_date_text), '%d.%m.%Y'),
    STR_TO_DATE(TRIM(o.delivery_date_text), '%d.%m.%Y'),
    CAST(TRIM(o.pickup_point_text) AS UNSIGNED),
    u.user_id,
    CAST(TRIM(o.pickup_code_text) AS UNSIGNED),

```

```

        st.status_id
FROM orders_import_raw o
LEFT JOIN users u
    ON u.full_name = TRIM(o.client_fio_text)
JOIN order_statuses st
    ON st.status_name = REPLACE(TRIM(o.status_text), '\r', '')
WHERE TRIM(o.order_number_text) <> '';

INSERT INTO order_items (order_id, product_id, quantity)
SELECT
    o.order_id,
    pr.product_id,
    CAST(TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(o.article_text, ',', 2), ',', -1))
AS UNSIGNED) AS qty
FROM orders o
JOIN products pr
    ON pr.article = TRIM(SUBSTRING_INDEX(o.article_text, ',', 1))
WHERE TRIM(o.article_text) <> ''

UNION ALL

SELECT
    o.order_id,
    pr.product_id,
    CAST(TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(o.article_text, ',', 4), ',', -1))
AS UNSIGNED) AS qty
FROM orders o
JOIN products pr
    ON pr.article = TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(o.article_text, ',', 3),
',', -1))
WHERE TRIM(o.article_text) <> '';

```

Шаг 6. Выполните контрольные SQL-проверки

Что делаем

Проверьте, что данные загружены полностью.

Команды/код

```
USE shoe2026_pu;
```

```

SELECT 'roles' AS table_name, COUNT(*) AS cnt FROM roles
UNION ALL SELECT 'users', COUNT(*) FROM users
UNION ALL SELECT 'categories', COUNT(*) FROM categories
UNION ALL SELECT 'manufacturers', COUNT(*) FROM manufacturers
UNION ALL SELECT 'suppliers', COUNT(*) FROM suppliers
UNION ALL SELECT 'products', COUNT(*) FROM products
UNION ALL SELECT 'pickup_points', COUNT(*) FROM pickup_points
UNION ALL SELECT 'order_statuses', COUNT(*) FROM order_statuses

```

```
UNION ALL SELECT 'orders', COUNT(*) FROM orders
UNION ALL SELECT 'order_items', COUNT(*) FROM order_items;
```

Должно получиться:

- roles = 4
- users = 10
- categories = 2
- manufacturers = 6
- suppliers = 2
- products = 30
- pickup_points = 36
- order_statuses = 2
- orders = 10
- order_items = 20

Для входа в приложение:

```
SELECT
    u.login,
    TRIM(REPLACE(u.password_plain, CHAR(13), '')) AS password_plain,
    u.full_name,
    r.role_name
FROM users u
JOIN roles r ON r.role_id = u.role_id
ORDER BY r.role_id, u.full_name;
```

Шаг 7. Создайте Python-проект и установите зависимости

Что делаем

Создайте виртуальное окружение и установите пакеты.

Команды/код

```
cd C:\shoe_store_2026_pu_python\app
python -m venv .venv
.\venv\Scripts\activate
pip install PyQt6 mysql-connector-python pyinstaller Pillow
pip freeze > requirements.txt
```

Шаг 8. Создайте db.py

Что делаем

Вынесите подключение к БД в отдельный модуль.

Команды/код

Создайте C:\shoe_store_2026_pu_python\app\db.py:

```
import mysql.connector

DB_CONFIG = {
    "host": "127.0.0.1",
    "port": 3306,
    "user": "demo",
    "password": "demo",
    "database": "shoe2026_pu",
    "use_unicode": True,
    "charset": "utf8mb4",
    "use_pure": True,
}
```

```
def get_connection():
    return mysql.connector.connect(**DB_CONFIG)
```

Если используете XAMPP/Open Server Panel/локальный MySQL Server, поменяйте параметры подключения под вашу локальную установку (часто root и пустой пароль).

Шаг 8.1. Создайте check_db.py и проверьте подключение

Что делаем

Проверьте соединение с БД через db.py.

Команды/код

Создайте C:\shoe_store_2026_pu_python\app\check_db.py:

```
from db import get_connection

def main():
    conn = None
    cur = None
    try:
        conn = get_connection()          # Проверяем, что параметры подключения
корректны
        cur = conn.cursor()
        cur.execute("SELECT 1")
        result = cur.fetchone()
        print(f"OK: подключение успешно, SELECT 1 -> {result[0]}")
    except Exception as e:
        print("ERROR:", e)
    finally:
        if cur:
            cur.close()
        if conn and conn.is_connected():
            conn.close()
```

```
if __name__ == "__main__":  
    main()
```

Запуск:

```
cd C:\shoe_store_2026_pu_python\app  
.\.venv\Scripts\activate  
python check_db.py
```

Шаг 9. Создайте main.py (финальная версия модулей 2–4)

Что делаем

Создайте сразу рабочую версию:

- окно входа (логин/пароль + гость);
- список товаров с ролями и оформлением;
- поиск/фильтр/сортировка для менеджера и администратора;
- добавление/редактирование/удаление товара для администратора;
- загрузка/замена фото товара (до 300x200) для администратора;
- окно заказов и форма заказа.

Команды/код

Создайте C:\shoe_store_2026_pu_python\app\main.py:

```
import os  
import shutil  
import sys  
import uuid  
from dataclasses import dataclass  
from datetime import datetime  
from decimal import Decimal  
  
from PyQt6.QtCore import Qt, QDate  
from PyQt6.QtGui import QColor, QFont, QIcon, QPixmap  
from PyQt6.QtWidgets import (  
    QApplication,  
    QComboBox,  
    QDateEdit,  
    QDialog,  
    QFileDialog,  
    QFormLayout,  
    QGridLayout,  
    QHBoxLayout,  
    QLabel,  
    QLineEdit,  
    QMainWindow,  
    QMessageBox,  
    QPushButton,
```

```

        QTableWidgetItem,
        QTableWidgetItem,
        QTextEdit,
        QVBoxLayout,
        QWidget,
    )

    from db import get_connection

    BASE_DIR = os.path.dirname(os.path.abspath(__file__))
    RESOURCES_DIR = os.path.join(os.path.dirname(BASE_DIR), "resources")
    PHOTOS_DIR = os.path.join(RESOURCES_DIR, "photos")
    ICON_ICO = os.path.join(RESOURCES_DIR, "Icon.ico")
    ICON_PNG = os.path.join(RESOURCES_DIR, "Icon.png")
    PICTURE_PNG = os.path.join(RESOURCES_DIR, "picture.png")

    COLOR_WHITE = "#FFFFFF"
    COLOR_SECOND = "#7FFF00"
    COLOR_ACCENT = "#00FA9A"
    COLOR_DISCOUNT_ROW = "#2E8B57"
    COLOR_ZERO_STOCK = "#ADD8E6"

    @dataclass
    class UserInfo:
        user_id: int | None
        full_name: str
        role_name: str

    def show_error(parent, text):
        QMessageBox.critical(parent, "Ошибка", text)

    def show_warn(parent, text):
        QMessageBox.warning(parent, "Предупреждение", text)

    class DataService:
        @staticmethod
        def auth(login: str, password: str):
            conn = get_connection()
            try:
                cur = conn.cursor(dictionary=True)
                cur.execute(
                    """
                    SELECT u.user_id, u.full_name, r.role_name
                    FROM users u
                    JOIN roles r ON r.role_id = u.role_id
                """
                )

```



```

        WHERE u.login = %s AND u.password_plain = %s
        """
        (login, password),
    )
    row = cur.fetchone()
    if not row:
        return None
    return UserInfo(row["user_id"], row["full_name"], row["role_name"])
finally:
    conn.close()

@staticmethod
def get_products():
    conn = get_connection()
    try:
        cur = conn.cursor(dictionary=True)
        cur.execute(
            """
            SELECT
                p.product_id,
                p.article,
                p.name,
                c.name AS category_name,
                p.description_text,
                m.name AS manufacturer_name,
                s.name AS supplier_name,
                p.price,
                p.discount_percent,
                p.unit_name,
                p.stock_quantity,
                p.photo_file
            FROM products p
            JOIN categories c ON c.category_id = p.category_id
            JOIN manufacturers m ON m.manufacturer_id = p.manufacturer_id
            JOIN suppliers s ON s.supplier_id = p.supplier_id
            ORDER BY p.product_id
            """
        )
        return cur.fetchall()
    finally:
        conn.close()

@staticmethod
def get_supplier_names():
    conn = get_connection()
    try:
        cur = conn.cursor()
        cur.execute("SELECT name FROM suppliers ORDER BY name")
        return [x[0] for x in cur.fetchall()]
    
```

```

        finally:
            conn.close()

    @staticmethod
    def get_categories():
        conn = get_connection()
        try:
            cur = conn.cursor(dictionary=True)
            cur.execute("SELECT category_id, name FROM categories ORDER BY name")
            return cur.fetchall()
        finally:
            conn.close()

    @staticmethod
    def get_manufacturers():
        conn = get_connection()
        try:
            cur = conn.cursor(dictionary=True)
            cur.execute("SELECT manufacturer_id, name FROM manufacturers ORDER BY
name")
            return cur.fetchall()
        finally:
            conn.close()

    @staticmethod
    def get_product_by_id(product_id: int):
        conn = get_connection()
        try:
            cur = conn.cursor(dictionary=True)
            cur.execute(
                """
                SELECT
                    p.product_id,
                    p.article,
                    p.name,
                    p.category_id,
                    p.description_text,
                    p.manufacturer_id,
                    s.name AS supplier_name,
                    p.price,
                    p.unit_name,
                    p.stock_quantity,
                    p.discount_percent,
                    p.photo_file
                FROM products p
                JOIN suppliers s ON s.supplier_id = p.supplier_id
                WHERE p.product_id = %s
                """,
                (product_id,),
            )

```

```

        )
        return cur.fetchone()
    finally:
        conn.close()

    @staticmethod
    def ensure_supplier(conn, name: str):
        cur = conn.cursor()
        cur.execute("SELECT supplier_id FROM suppliers WHERE name=%s", (name,))
        row = cur.fetchone()
        if row:
            return row[0]
        cur.execute("INSERT INTO suppliers(name) VALUES(%s)", (name,))
        conn.commit()
        return cur.lastrowid

    @staticmethod
    def save_product(model: dict):
        conn = get_connection()
        try:
            supplier_id = DataService.ensure_supplier(conn,
model["supplier_name"])
            cur = conn.cursor()
            if model.get("product_id"):
                cur.execute(
                    """
                    UPDATE products
                    SET
                        article=%s,
                        name=%s,
                        category_id=%s,
                        description_text=%s,
                        manufacturer_id=%s,
                        supplier_id=%s,
                        price=%s,
                        unit_name=%s,
                        stock_quantity=%s,
                        discount_percent=%s,
                        photo_file=%s
                    WHERE product_id=%s
                    """,
                    (
                        model["article"],
                        model["name"],
                        model["category_id"],
                        model["description_text"],
                        model["manufacturer_id"],
                        supplier_id,
                        model["price"],

```

```

        model["unit_name"],
        model["stock_quantity"],
        model["discount_percent"],
        model["photo_file"],
        model["product_id"],
    ),
)
else:
    cur.execute(
        """
        INSERT INTO products(
            article, name, category_id, description_text,
            manufacturer_id, supplier_id, price, unit_name,
            stock_quantity, discount_percent, photo_file
        )
        VALUES(%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)
        """,
        (
            model["article"],
            model["name"],
            model["category_id"],
            model["description_text"],
            model["manufacturer_id"],
            supplier_id,
            model["price"],
            model["unit_name"],
            model["stock_quantity"],
            model["discount_percent"],
            model["photo_file"],
        ),
    )
    conn.commit()
finally:
    conn.close()

@staticmethod
def product_exists_in_orders(product_id: int) -> bool:
    conn = get_connection()
    try:
        cur = conn.cursor()
        cur.execute("SELECT COUNT(*) FROM order_items WHERE product_id=%s",
(product_id,))
        return cur.fetchone()[0] > 0
    finally:
        conn.close()

@staticmethod
def delete_product(product_id: int):
    conn = get_connection()

```

```

        try:
            cur = conn.cursor()
            cur.execute("DELETE FROM products WHERE product_id=%s",
(product_id,))
            conn.commit()
        finally:
            conn.close()

    @staticmethod
    def get_orders():
        conn = get_connection()
        try:
            cur = conn.cursor(dictionary=True)
            cur.execute(
                """
                SELECT
                    o.order_id,
                    o.order_number,
                    o.article_text,
                    os.status_name,
                    pp.address_text,
                    o.order_date,
                    o.delivery_date
                FROM orders o
                JOIN order_statuses os ON os.status_id = o.status_id
                JOIN pickup_points pp ON pp.pickup_point_id = o.pickup_point_id
                ORDER BY o.order_number
                """
            )
            return cur.fetchall()
        finally:
            conn.close()

    @staticmethod
    def get_order_statuses():
        conn = get_connection()
        try:
            cur = conn.cursor(dictionary=True)
            cur.execute("SELECT status_id, status_name FROM order_statuses ORDER
BY status_name")
            return cur.fetchall()
        finally:
            conn.close()

    @staticmethod
    def get_pickup_points():
        conn = get_connection()
        try:
            cur = conn.cursor(dictionary=True)

```

```

        cur.execute("SELECT pickup_point_id, address_text FROM pickup_points
ORDER BY pickup_point_id")
        return cur.fetchall()
    finally:
        conn.close()

    @staticmethod
    def get_order_by_id(order_id: int):
        conn = get_connection()
        try:
            cur = conn.cursor(dictionary=True)
            cur.execute("SELECT * FROM orders WHERE order_id=%s", (order_id,))
            return cur.fetchone()
        finally:
            conn.close()

    @staticmethod
    def get_next_order_number():
        conn = get_connection()
        try:
            cur = conn.cursor()
            cur.execute("SELECT COALESCE(MAX(order_number), 0) + 1 FROM orders")
            return cur.fetchone()[0]
        finally:
            conn.close()

    @staticmethod
    def get_next_pickup_code():
        conn = get_connection()
        try:
            cur = conn.cursor()
            cur.execute("SELECT COALESCE(MAX(pickup_code), 900) + 1 FROM orders")
            return cur.fetchone()[0]
        finally:
            conn.close()

    @staticmethod
    def parse_article_pairs(text: str):
        tokens = [x.strip() for x in (text or "").split(",") if x.strip()]
        if len(tokens) < 2 or len(tokens) % 2 != 0:
            raise ValueError("Артикулы задаются парами: артикул, количество.")
        pairs = []
        for i in range(0, len(tokens), 2):
            article = tokens[i]
            try:
                qty = int(tokens[i + 1])
            except Exception:
                raise ValueError("Количество в паре артикулов должно быть целым
числом.")

```

```

        if qty <= 0:
            raise ValueError("Количество в паре артикулов должно быть больше
0.")
        pairs.append((article, qty))
    return pairs

    @staticmethod
    def save_order(model: dict):
        conn = get_connection()
        try:
            pairs = DataService.parse_article_pairs(model["article_text"])
            cur = conn.cursor()

            article_ids = {}
            for article, _ in pairs:
                cur.execute("SELECT product_id FROM products WHERE article=%s",
(article,))
                row = cur.fetchone()
                if not row:
                    raise ValueError(f"Артикул {article} не найден в таблице
products.")
                article_ids[article] = row[0]

            if model.get("order_id"):
                cur.execute(
                    """
                    UPDATE orders
                    SET
                        article_text=%s,
                        order_date=%s,
                        delivery_date=%s,
                        pickup_point_id=%s,
                        status_id=%s
                    WHERE order_id=%s
                    """,
                    (
                        model["article_text"],
                        model["order_date"],
                        model["delivery_date"],
                        model["pickup_point_id"],
                        model["status_id"],
                        model["order_id"],
                    ),
                )
                order_id = model["order_id"]
                cur.execute("DELETE FROM order_items WHERE order_id=%s",
(order_id,))
            else:
                cur.execute(

```

```

        """
        INSERT INTO orders(
            order_number, article_text, order_date, delivery_date,
            pickup_point_id, client_user_id, pickup_code, status_id
        )
        VALUES(%s,%s,%s,%s,%s,%s,%s,%s)
        """
        (
            model["order_number"],
            model["article_text"],
            model["order_date"],
            model["delivery_date"],
            model["pickup_point_id"],
            None,
            model["pickup_code"],
            model["status_id"],
        ),
    )
    order_id = cur.lastrowid

    for article, qty in pairs:
        cur.execute(
            "INSERT INTO order_items(order_id, product_id, quantity)
VALUES(%s,%s,%s)",
            (order_id, article_ids[article], qty),
        )

    conn.commit()
finally:
    conn.close()

@staticmethod
def delete_order(order_id: int):
    conn = get_connection()
    try:
        cur = conn.cursor()
        cur.execute("DELETE FROM orders WHERE order_id=%s", (order_id,))
        conn.commit()
    finally:
        conn.close()

class LoginDialog(QDialog):
    def __init__(self):
        super().__init__()
        self.user_data = None

        self.setWindowTitle("Bxog")
        self.setMinimumWidth(430)

```



```

if os.path.exists(ICON_ICO):
    self.setWindowIcon(QIcon(ICON_ICO))

root = QVBoxLayout(self)
form = QFormLayout()

self.login_edit = QLineEdit()
self.password_edit = QLineEdit()
self.password_edit.setEchoMode(QLineEdit.EchoMode.Password)
form.addRow("Логин:", self.login_edit)
form.addRow("Пароль:", self.password_edit)
root.addLayout(form)

buttons = QHBoxLayout()
btn_login = QPushButton("Войти")
btn_guest = QPushButton("Войти как гость")
btn_login.setStyleSheet(f"background:{COLOR_ACCENT};")
btn_guest.setStyleSheet(f"background:{COLOR_ACCENT};")
btn_login.clicked.connect(self.login_click)
btn_guest.clicked.connect(self.guest_click)
buttons.addWidget(btn_login)
buttons.addWidget(btn_guest)
root.addLayout(buttons)

def login_click(self):
    login = self.login_edit.text().strip()
    password = self.password_edit.text().strip()
    if not login or not password:
        show_warn(self, "Введите логин и пароль.")
        return
    try:
        user = DataService.auth(login, password)
    except Exception as ex:
        show_error(self, f"Ошибка подключения к БД:\n{ex}")
        return
    if user is None:
        show_error(self, "Неверный логин или пароль.")
        return
    self.user_data = user
    self.accept()

def guest_click(self):
    self.user_data = UserInfo(None, "Гость", "Гость")
    self.accept()

class ProductFormDialog(QDialog):
    def __init__(self, product_id=None):
        super().__init__()

```

```

self.product_id = product_id
self.old_photo_file = ""
self.selected_photo_path = ""

self.setWindowTitle("Добавление/редактирование товара")
self.setMinimumWidth(760)
if os.path.exists(ICON_ICO):
    self.setWindowIcon(QIcon(ICON_ICO))

root = QVBoxLayout(self)
grid = QGridLayout()
root.addLayout(grid)

row = 0
self.id_label = QLabel("ID товара:")
self.id_edit = QLineEdit()
self.id_edit.setReadOnly(True)
grid.addWidget(self.id_label, row, 0)
grid.addWidget(self.id_edit, row, 1)
row += 1

self.article_edit = QLineEdit()
self.name_edit = QLineEdit()
self.category_combo = QComboBox()
self.description_edit = QTextEdit()
self.description_edit.setFixedHeight(90)
self.manufacturer_combo = QComboBox()
self.supplier_edit = QLineEdit()
self.price_edit = QLineEdit()
self.unit_edit = QLineEdit()
self.stock_edit = QLineEdit()
self.discount_edit = QLineEdit()

fields = [
    ("Артикул:", self.article_edit),
    ("Наименование товара:", self.name_edit),
    ("Категория товара:", self.category_combo),
    ("Описание товара:", self.description_edit),
    ("Производитель:", self.manufacturer_combo),
    ("Поставщик:", self.supplier_edit),
    ("Цена:", self.price_edit),
    ("Единица измерения:", self.unit_edit),
    ("Количество на складе:", self.stock_edit),
    ("Действующая скидка (%):", self.discount_edit),
]

for title, widget in fields:
    grid.addWidget(QLabel(title), row, 0)
    grid.addWidget(widget, row, 1)
    row += 1

```

```

photo_row = QHBoxLayout()
self.photo_label = QLabel()
self.photo_label.setFixedSize(300, 200)
self.photo_label.setStyleSheet("border:1px solid #546F94;")
self.photo_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
btn_photo = QPushButton("Выбрать фото")
btn_photo.clicked.connect(self.choose_photo)
photo_row.addWidget(self.photo_label)
photo_row.addWidget(btn_photo)
root.addLayout(photo_row)

buttons = QHBoxLayout()
btn_save = QPushButton("Сохранить")
btn_back = QPushButton("Назад")
btn_save.setStyleSheet(f"background:{COLOR_ACCENT};")
btn_back.setStyleSheet(f"background:{COLOR_SECOND};")
btn_save.clicked.connect(self.save_click)
btn_back.clicked.connect(self.reject)
buttons.addWidget(btn_save)
buttons.addWidget(btn_back)
root.addLayout(buttons)

self.load_lookups()

if self.product_id:
    self.load_product(self.product_id)
else:
    self.id_label.hide()
    self.id_edit.hide()
    self.set_preview(PICTURE_PNG)

def load_lookups(self):
    for x in DataService.get_categories():
        self.category_combo.addItem(x["name"], x["category_id"])
    for x in DataService.get_manufacturers():
        self.manufacturer_combo.addItem(x["name"], x["manufacturer_id"])

def resolve_photo_path(self, photo_file: str):
    if photo_file:
        file_name = os.path.basename(photo_file.strip())
        p = os.path.join(PHOTOS_DIR, file_name)
        if os.path.exists(p):
            return p
    return PICTURE_PNG

def load_product(self, product_id: int):
    row = DataService.get_product_by_id(product_id)
    if not row:

```

```

        show_error(self, "Товар не найден.")
        self.reject()
        return

    self.id_edit.setText(str(row["product_id"]))
    self.article_edit.setText(row["article"] or "")
    self.name_edit.setText(row["name"] or "")
    self.description_edit.setPlainText(row["description_text"] or "")
    self.supplier_edit.setText(row["supplier_name"] or "")
    self.price_edit.setText(str(row["price"]))
    self.unit_edit.setText(row["unit_name"] or "")
    self.stock_edit.setText(str(row["stock_quantity"]))
    self.discount_edit.setText(str(row["discount_percent"]))

    i = self.category_combo.findData(row["category_id"])
    if i >= 0:
        self.category_combo.setCurrentIndex(i)
    j = self.manufacturer_combo.findData(row["manufacturer_id"])
    if j >= 0:
        self.manufacturer_combo.setCurrentIndex(j)

    self.old_photo_file = row["photo_file"] or ""
    self.set_preview(self.resolve_photo_path(self.old_photo_file))

def set_preview(self, file_path: str):
    pix = QPixmap(file_path)
    if pix.isNull():
        self.photo_label.setText("Нет фото")
        return
    self.photo_label.setPixmap(
        pix.scaled(300, 200, Qt.AspectRatioMode.KeepAspectRatio,
Qt.TransformationMode.SmoothTransformation)
    )

def choose_photo(self):
    path, _ = QFileDialog.getOpenFileName(
        self,
        "Выберите фото",
        "",
        "Images (*.png *.jpg *.jpeg *.bmp)",
    )
    if not path:
        return
    pix = QPixmap(path)
    if pix.isNull():
        show_warn(self, "Не удалось прочитать изображение.")
        return
    if pix.width() > 300 or pix.height() > 200:
        show_warn(self, "Размер фото не должен превышать 300x200 пикселей.")

```

```

        return
    self.selected_photo_path = path
    self.set_preview(path)

def save_click(self):
    article = self.article_edit.text().strip()
    name = self.name_edit.text().strip()
    supplier = self.supplier_edit.text().strip()
    unit = self.unit_edit.text().strip()
    description = self.description_edit.toPlainText().strip()

    if not article or not name or not supplier or not unit:
        show_warn(self, "Заполните обязательные поля: артикул, наименование, поставщик, единица измерения.")
        return

    try:
        price = Decimal(self.price_edit.text().strip().replace(",", "."))
        stock = int(self.stock_edit.text().strip())
        discount = Decimal(self.discount_edit.text().strip().replace(",",
"."))
    except Exception:
        show_error(self, "Проверьте формат цены, количества и скидки.")
        return

    if price < 0 or stock < 0 or discount < 0:
        show_error(self, "Цена, количество и скидка не могут быть отрицательными.")
        return

    photo_file = self.old_photo_file

    # Неразрывная логика замены фото: копируем новое и удаляем старое.
    if self.selected_photo_path:
        os.makedirs(PHOTOS_DIR, exist_ok=True)
        ext = os.path.splitext(self.selected_photo_path)[1].lower() or ".png"
        new_name = f"uploaded_{uuid.uuid4().hex}{ext}"
        target = os.path.join(PHOTOS_DIR, new_name)
        shutil.copy2(self.selected_photo_path, target)
        photo_file = f"resources/photos/{new_name}"

    if self.old_photo_file:
        old_name = os.path.basename(self.old_photo_file)
        old_path = os.path.join(PHOTOS_DIR, old_name)
        if os.path.exists(old_path):
            try:
                os.remove(old_path)
            except Exception:
                pass

```

```

model = {
    "product_id": self.product_id,
    "article": article,
    "name": name,
    "category_id": self.category_combo.currentData(),
    "description_text": description or None,
    "manufacturer_id": self.manufacturer_combo.currentData(),
    "supplier_name": supplier,
    "price": price,
    "unit_name": unit,
    "stock_quantity": stock,
    "discount_percent": discount,
    "photo_file": photo_file or None,
}

try:
    DataService.save_product(model)
except Exception as ex:
    show_error(self, f"Ошибка сохранения:\n{ex}")
    return

self.accept()

class OrderFormDialog(QDialog):
    def __init__(self, order_id=None):
        super().__init__()
        self.order_id = order_id

        self.setWindowTitle("Добавление/редактирование заказа")
        self.setMinimumWidth(680)
        if os.path.exists(ICON_ICO):
            self.setWindowIcon(QIcon(ICON_ICO))

        root = QVBoxLayout(self)
        form = QFormLayout()
        root.addLayout(form)

        self.order_number_edit = QLineEdit()
        self.order_number_edit.setReadOnly(True)
        self.articles_edit = QLineEdit()
        self.status_combo = QComboBox()
        self.pickup_combo = QComboBox()
        self.order_date_edit = QDateEdit()
        self.order_date_edit.setCalendarPopup(True)
        self.order_date_edit.setDisplayFormat("yyyy-MM-dd")
        self.delivery_date_edit = QDateEdit()
        self.delivery_date_edit.setCalendarPopup(True)

```

```

self.delivery_date_edit.setDisplayFormat("yyyy-MM-dd")

form.addRow("Номер заказа:", self.order_number_edit)
form.addRow("Артикул:", self.articles_edit)
form.addRow("Статус заказа:", self.status_combo)
form.addRow("Адрес пункта выдачи:", self.pickup_combo)
form.addRow("Дата заказа:", self.order_date_edit)
form.addRow("Дата выдачи:", self.delivery_date_edit)

for x in DataService.get_order_statuses():
    self.status_combo.addItem(x["status_name"], x["status_id"])
for x in DataService.get_pickup_points():
    self.pickup_combo.addItem(f'{x["pickup_point_id"]}. {x["address_text"]}', x["pickup_point_id"])

if self.order_id:
    self.load_order(self.order_id)
else:
    self.order_number_edit.setText(str(DataService.get_next_order_number()))
    code = DataService.get_next_pickup_code()
    self.generated_pickup_code = code
    today = QDate.currentDate()
    self.order_date_edit.setDate(today)
    self.delivery_date_edit.setDate(today.addDays(1))

buttons = QHBoxLayout()
btn_save = QPushButton("Сохранить")
btn_back = QPushButton("Назад")
btn_save.setStyleSheet(f"background:{COLOR_ACCENT};")
btn_back.setStyleSheet(f"background:{COLOR_SECOND};")
btn_save.clicked.connect(self.save_click)
btn_back.clicked.connect(self.reject)
buttons.addWidget(btn_save)
buttons.addWidget(btn_back)
root.addLayout(buttons)

def load_order(self, order_id: int):
    row = DataService.get_order_by_id(order_id)
    if not row:
        show_error(self, "Заказ не найден.")
        self.reject()
        return

    self.order_number_edit.setText(str(row["order_number"]))
    self.articles_edit.setText(row["article_text"] or "")
    self.generated_pickup_code = int(row["pickup_code"])

    i = self.status_combo.findData(row["status_id"])
    if i >= 0:

```

```

        self.status_combo.setCurrentIndex(i)

j = self.pickup_combo.findData(row["pickup_point_id"])
if j >= 0:
    self.pickup_combo.setCurrentIndex(j)

if row["order_date"]:
    dt = row["order_date"]
    self.order_date_edit.setDate(QDate(dt.year, dt.month, dt.day))
if row["delivery_date"]:
    dt = row["delivery_date"]
    self.delivery_date_edit.setDate(QDate(dt.year, dt.month, dt.day))

def save_click(self):
    articles = self.articles_edit.text().strip()
    if not articles:
        show_warn(self, "Поле Артикул обязательно.")
        return

    try:
        DataService.parse_article_pairs(articles)
    except Exception as ex:
        show_error(self, str(ex))
        return

    model = {
        "order_id": self.order_id,
        "order_number": int(self.order_number_edit.text()),
        "article_text": articles,
        "status_id": self.status_combo.currentData(),
        "pickup_point_id": self.pickup_combo.currentData(),
        "order_date": self.order_date_edit.date().toString("yyyy-MM-dd"),
        "delivery_date": self.delivery_date_edit.date().toString("yyyy-MM-
dd"),
        "pickup_code": self.generated_pickup_code,
    }

    try:
        DataService.save_order(model)
    except Exception as ex:
        show_error(self, f"Ошибка сохранения:\n{ex}")
        return

    self.accept()

class OrdersDialog(QDialog):
    def __init__(self, user_data: UserInfo, parent=None):
        super().__init__(parent)

```



```

self.user_data = user_data

self.setWindowTitle("Заказы")
self.resize(1200, 640)
if os.path.exists(ICON_ICO):
    self.setWindowIcon(QIcon(ICON_ICO))

root = QVBoxLayout(self)

top = QHBoxLayout()
title = QLabel("Список заказов")
title.setStyleSheet("font-size:24px; font-weight:700;")
top.addWidget(title)
top.addStretch()

self.btn_add = QPushButton("Добавить заказ")
self.btn_edit = QPushButton("Редактировать заказ")
self.btn_delete = QPushButton("Удалить заказ")
btn_back = QPushButton("Назад")
self.btn_add.clicked.connect(self.add_order)
self.btn_edit.clicked.connect(self.edit_order)
self.btn_delete.clicked.connect(self.delete_order)
btn_back.clicked.connect(self.close)
top.addWidget(self.btn_add)
top.addWidget(self.btn_edit)
top.addWidget(self.btn_delete)
top.addWidget(btn_back)
root.addLayout(top)

self.table = QTableWidget()
self.table.setColumnCount(6)
self.table.setHorizontalHeaderLabels(
    ["Номер", "Артикул заказа", "Статус", "Пункт выдачи", "Дата заказа",
"Дата выдачи"]
)
self.table.doubleClicked.connect(self.double_click_order)
root.addWidget(self.table)

is_admin = self.user_data.role_name == "Администратор"
self.btn_add.setVisible(is_admin)
self.btn_edit.setVisible(is_admin)
self.btn_delete.setVisible(is_admin)

self.load_orders()

def load_orders(self):
    rows = DataService.get_orders()
    self.table.setRowCount(len(rows))
    for i, x in enumerate(rows):

```

```

        self.table.setItem(i, 0, QTableWidgetItem(str(x["order_number"])))
        self.table.setItem(i, 1, QTableWidgetItem(x["article_text"] or ""))
        self.table.setItem(i, 2, QTableWidgetItem(x["status_name"] or ""))
        self.table.setItem(i, 3, QTableWidgetItem(x["address_text"] or ""))
        self.table.setItem(i, 4, QTableWidgetItem(str(x["order_date"] or
"")))
        self.table.setItem(i, 5, QTableWidgetItem(str(x["delivery_date"] or
"")))
        self.table.item(i, 0).setData(Qt.ItemDataRole.UserRole,
x["order_id"])
        self.table.resizeColumnsToContents()

def current_order_id(self):
    row = self.table.currentRow()
    if row < 0:
        return None
    item = self.table.item(row, 0)
    if item is None:
        return None
    return item.data(Qt.ItemDataRole.UserRole)

def add_order(self):
    if self.user_data.role_name != "Администратор":
        return
    dlg = OrderFormDialog(None)
    if dlg.exec():
        self.load_orders()

def edit_order(self):
    if self.user_data.role_name != "Администратор":
        return
    order_id = self.current_order_id()
    if not order_id:
        show_warn(self, "Выберите заказ для редактирования.")
        return
    dlg = OrderFormDialog(order_id)
    if dlg.exec():
        self.load_orders()

def double_click_order(self):
    if self.user_data.role_name == "Администратор":
        self.edit_order()

def delete_order(self):
    if self.user_data.role_name != "Администратор":
        return
    order_id = self.current_order_id()
    if not order_id:
        show_warn(self, "Выберите заказ для удаления.")

```

```

        return
    ans = QMessageBox.question(self, "Подтверждение", "Удалить выбранный
заказ?")
    if ans != QMessageBox.StandardButton.Yes:
        return
    try:
        DataService.delete_order(order_id)
    except Exception as ex:
        show_error(self, f"Ошибка удаления:\n{ex}")
    return
self.load_orders()

class ProductsWindow(QMainWindow):
    def __init__(self, user_data: UserInfo):
        super().__init__()
        self.user_data = user_data
        self.products = []
        self.product_editor_opened = False

        self.setWindowTitle("Список товаров")
        self.resize(1600, 820)
        if os.path.exists(ICON_ICO):
            self.setWindowIcon(QIcon(ICON_ICO))

        root = QWidget()
        self.setCentralWidget(root)
        main = QVBoxLayout(root)

        header = QGridLayout()
        logo = QLabel()
        if os.path.exists(ICON_PNG):
            pix = QPixmap(ICON_PNG).scaled(120, 70,
Qt.AspectRatioMode.KeepAspectRatio, Qt.TransformationMode.SmoothTransformation)
            logo.setPixmap(pix)
        title = QLabel("ООО «Обувь» – Список товаров")
        title.setStyleSheet("font-size:28px; font-weight:700;")
        self.role_label = QLabel(f"Роль:
{self.role_caption(self.user_data.role_name)}")
        self.user_label = QLabel(f"Пользователь: {self.user_data.full_name}")
        self.btn_orders = QPushButton("Заказы")
        self.btn_orders.setStyleSheet(f"background:{COLOR_SECOND};")
        self.btn_orders.clicked.connect(self.open_orders)
        btn_logout = QPushButton("Выход")
        btn_logout.setStyleSheet(f"background:{COLOR_ACCENT};")
        btn_logout.clicked.connect(self.logout)

        right = QVBoxLayout()
        right.addWidget(self.role_label, alignment=Qt.AlignmentFlag.AlignRight)

```

```

right.addWidget(self.user_label, alignment=Qt.AlignmentFlag.AlignRight)
right.addWidget(self.btn_orders, alignment=Qt.AlignmentFlag.AlignRight)
right.addWidget(btn_logout, alignment=Qt.AlignmentFlag.AlignRight)

header.addWidget(logo, 0, 0)
header.addWidget(title, 0, 1)
header.addLayout(right, 0, 2)
header.setColumnStretch(1, 1)
main.addLayout(header)

self.filter_wrap = QWidget()
filter_layout = QHBoxLayout(self.filter_wrap)
filter_layout.addWidget(QLabel("Поиск:"))
self.search_edit = QLineEdit()
filter_layout.addWidget(self.search_edit)
filter_layout.addWidget(QLabel("Поставщик:"))
self.supplier_combo = QComboBox()
filter_layout.addWidget(self.supplier_combo)
filter_layout.addWidget(QLabel("Сортировка:"))
self.sort_combo = QComboBox()
self.sort_combo.addItem(["Без сортировки", "Остаток (по возрастанию)",
"Остаток (по убыванию)"])
filter_layout.addWidget(self.sort_combo)
main.addWidget(self.filter_wrap)

self.admin_wrap = QWidget()
admin_layout = QHBoxLayout(self.admin_wrap)
self.btn_add = QPushButton("Добавить товар")
self.btn_edit = QPushButton("Редактировать товар")
self.btn_delete = QPushButton("Удалить товар")
self.btn_add.setStyleSheet(f"background:{COLOR_ACCENT};")
self.btn_edit.setStyleSheet(f"background:{COLOR_SECOND};")
self.btn_delete.setStyleSheet(f"background:{COLOR_SECOND};")
admin_layout.addWidget(self.btn_add)
admin_layout.addWidget(self.btn_edit)
admin_layout.addWidget(self.btn_delete)
admin_layout.addStretch()
main.addWidget(self.admin_wrap)

self.table = QTableWidgetItem()
self.table.setColumnCount(12)
self.table.setHorizontalHeaderLabels(
    [
        "Фото",
        "Артикул",
        "Наименование",
        "Категория",
        "Описание",
        "Производитель",

```

```

        "Поставщик",
        "Цена",
        "Цена со скидкой",
        "Ед.",
        "Остаток",
        "Скидка",
    ]
)

self.table.setSelectionBehavior(QTableWidget.SelectionBehavior.SelectRows)
self.table.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)
self.table.doubleClicked.connect(self.row_double_click)
main.addWidget(self.table)

self.btn_add.clicked.connect(self.add_product)
self.btn_edit.clicked.connect(self.edit_product)
self.btn_delete.clicked.connect(self.delete_product)
self.search_edit.textChanged.connect(self.apply_filters)
self.supplier_combo.currentTextChanged.connect(self.apply_filters)
self.sort_combo.currentTextChanged.connect(self.apply_filters)

self.apply_role_rules()
self.load_suppliers()
self.load_products()

def role_caption(self, role_name: str):
    if role_name == "Авторизированный клиент":
        return "Клиент"
    return role_name

def is_admin(self):
    return self.user_data.role_name == "Администратор"

def is_manager_or_admin(self):
    return self.user_data.role_name in ("Менеджер", "Администратор")

def apply_role_rules(self):
    advanced = self.is_manager_or_admin()
    self.filter_wrap.setVisible(advanced)
    self.admin_wrap.setVisible(self.is_admin())
    self.btn_orders.setVisible(advanced)

def load_suppliers(self):
    self.supplier_combo.clear()
    self.supplier_combo.addItem("Все поставщики")
    for x in DataService.get_supplier_names():
        self.supplier_combo.addItem(x)

def load_products(self):
    self.products = DataService.get_products()

```

```

        self.apply_filters()

def photo_pixmap(self, photo_file: str):
    if photo_file:
        p = os.path.join(PHOTOS_DIR, os.path.basename(photo_file))
        if os.path.exists(p):
            pix = QPixmap(p)
            if not pix.isNull():
                return pix
    pix = QPixmap(PICTURE_PNG)
    return pix

def apply_filters(self):
    data = list(self.products)

    if self.is_manager_or_admin():
        text = self.search_edit.text().strip().lower()
        supplier = self.supplier_combo.currentText()
        sort_mode = self.sort_combo.currentText()

        if supplier and supplier != "Все поставщики":
            data = [x for x in data if (x.get("supplier_name") or "").lower()
== supplier.lower()]

        if text:
            def hit(row):
                fields = [
                    row.get("article"),
                    row.get("name"),
                    row.get("category_name"),
                    row.get("description_text"),
                    row.get("manufacturer_name"),
                    row.get("supplier_name"),
                    row.get("unit_name"),
                ]
                return any(text in str(v or "").lower() for v in fields)
            data = [x for x in data if hit(x)]

        if sort_mode == "Остаток (по возрастанию)":
            data.sort(key=lambda r: int(r.get("stock_quantity") or 0))
        elif sort_mode == "Остаток (по убыванию)":
            data.sort(key=lambda r: int(r.get("stock_quantity") or 0),
reverse=True)

        self.fill_table(data)

def fill_table(self, rows):
    self.table.setRowCount(len(rows))
    for i, row in enumerate(rows):

```

```

        pix = self.photo_pixmap(row.get("photo_file") or "")
        img_label = QLabel()
        img_label.setAlignment(Qt.AlignmentFlag.AlignCenter)
        img_label.setPixmap(pix.scaled(90, 60,
Qt.AspectRatioMode.KeepAspectRatio, Qt.TransformationMode.SmoothTransformation))
        self.table.setCellWidget(i, 0, img_label)

        price = Decimal(str(row.get("price") or 0))
        discount = Decimal(str(row.get("discount_percent") or 0))
        final_price = (price * (Decimal("100") - discount) /
Decimal("100")).quantize(Decimal("0.01"))

        values = [
            row.get("article") or "",
            row.get("name") or "",
            row.get("category_name") or "",
            row.get("description_text") or "",
            row.get("manufacturer_name") or "",
            row.get("supplier_name") or "",
            f"{price:.2f}",
            f"{final_price:.2f}",
            row.get("unit_name") or "",
            str(row.get("stock_quantity") or 0),
            f"{discount:.2f}",
        ]
        for c, v in enumerate(values, start=1):
            item = QTableWidgetItem(v)
            self.table.setItem(i, c, item)

        # Храним product_id в первой текстовой колонке строки.
        self.table.item(i, 1).setData(Qt.ItemDataRole.UserRole,
row["product_id"])

        price_item = self.table.item(i, 7)
        if discount > 0:
            f = price_item.font()
            f.setStrikeOut(True)
            price_item.setFont(f)
            price_item.setForeground(QColor("red"))

        row_color = QColor(COLOR_WHITE)
        fg_color = QColor("#111827")
        if int(row.get("stock_quantity") or 0) == 0:
            row_color = QColor(COLOR_ZERO_STOCK)
        elif discount > 15:
            row_color = QColor(COLOR_DISCOUNT_ROW)

        for col in range(1, self.table.columnCount()):
            cell = self.table.item(i, col)

```

```

        if cell:
            cell.setBackground(row_color)
            cell.setForeground(fg_color)

    self.table.resizeColumnsToContents()
    self.table.setColumnWidth(0, 100)

def selected_product_id(self):
    row = self.table.currentRow()
    if row < 0:
        return None
    item = self.table.item(row, 1)
    if item is None:
        return None
    return item.data(Qt.ItemDataRole.UserRole)

def add_product(self):
    if not self.is_admin():
        return
    if self.product_editor_opened:
        show_warn(self, "Окно редактирования уже открыто.")
        return
    self.product_editor_opened = True
    dlg = ProductFormDialog(None)
    ok = dlg.exec()
    self.product_editor_opened = False
    if ok:
        self.load_suppliers()
        self.load_products()

def edit_product(self):
    if not self.is_admin():
        return
    product_id = self.selected_product_id()
    if not product_id:
        show_warn(self, "Выберите товар для редактирования.")
        return
    if self.product_editor_opened:
        show_warn(self, "Окно редактирования уже открыто.")
        return
    self.product_editor_opened = True
    dlg = ProductFormDialog(product_id)
    ok = dlg.exec()
    self.product_editor_opened = False
    if ok:
        self.load_suppliers()
        self.load_products()

def row_double_click(self):

```



```

        if self.is_admin():
            self.edit_product()

def delete_product(self):
    if not self.is_admin():
        return
    product_id = self.selected_product_id()
    if not product_id:
        show_warn(self, "Выберите товар для удаления.")
        return
    if DataService.product_exists_in_orders(product_id):
        show_warn(self, "Товар присутствует в заказе. Удаление невозможно.")
        return
    ans = QMessageBox.question(self, "Подтверждение", "Удалить выбранный
товар?")
    if ans != QMessageBox.StandardButton.Yes:
        return
    try:
        DataService.delete_product(product_id)
    except Exception as ex:
        show_error(self, f"Ошибка удаления:\n{ex}")
        return
    self.load_suppliers()
    self.load_products()

def open_orders(self):
    if not self.is_manager_or_admin():
        return
    dlg = OrdersDialog(self.user_data, self)
    dlg.exec()

def logout(self):
    self.close()
    login = LoginDialog()
    if login.exec():
        self.next_window = ProductsWindow(login.user_data)
        self.next_window.show()

def main():
    app = QApplication(sys.argv)
    app.setStyle("Fusion")
    app.setFont(QFont("Times New Roman", 11))

    login = LoginDialog()
    if not login.exec():
        sys.exit(0)

    window = ProductsWindow(login.user_data)

```

```
window.show()
sys.exit(app.exec())

if __name__ == "__main__":
    main()
```

Шаг 10. Запустите приложение и проверьте роли

Что делаем

Проверьте вход под гостем и под учетными записями из БД.

Команды/код

```
cd C:\shoe_store_2026_pu_python\app
.\.venv\Scripts\activate
python main.py
```

Проверьте:

- вход как гость и вход под пользователем из users;
 - ФИО в правом верхнем углу;
 - Выход возвращает в окно входа;
 - у гостя/клиента: только просмотр товаров;
 - у менеджера: поиск, фильтр, сортировка и просмотр заказов;
 - у администратора: CRUD товаров, CRUD заказов.
-

Шаг 11. Проверьте модули 2-4

Что делаем

Проверьте ключевые требования задания.

Команды/код

1. Список товаров:

- если discount_percent > 15, строка #2E8B57;
- если скидка > 0, базовая цена зачеркнута красным, итоговая отображается рядом;
- если stock_quantity = 0, строка голубая;
- если фото отсутствует, используется picture.png.

2. Фильтрация/поиск/сортировка:

- фильтр по поставщику (первый пункт: Все поставщики);
- поиск по всем текстовым полям;
- сортировка по остатку вверх/вниз;
- поиск + фильтр работают совместно, без кнопки «Найти».

3. Форма товара:

- добавление и редактирование доступны только администратору;
- при добавлении ID не показывается, при редактировании ID только для чтения;
- можно выбрать фото (до 300x200);
- при замене фото старое удаляется из папки приложения;
- нельзя открыть второе окно редактирования, пока открыто первое.

4. Удаление товара:

- если товар есть в order_items, удаление запрещено.

5. Заказы:

- кнопка Заказы есть у менеджера и администратора;
- менеджер: только просмотр;
- администратор: добавление/редактирование/удаление;
- после изменений список заказов обновляется.

Шаг 11.1. Зафиксируйте локальный git-коммит (модули 2-4)

Что делаем

Сделайте локальный коммит после реализации функционала модулей 2–4.

Команды/код

```
cd C:\shoe_store_2026_ru_python
git init
git add app
git commit -m "Реализованы модули 2-4 (Python, Вариант 1 2026)"
```

Шаг 12. Подготовьте блок-схему алгоритма по ГОСТ и сохраните PDF

Что делаем

Оформите блок-схему алгоритма разработки приложения согласно ГОСТ 19.701-90.

Команды/код

1. Откройте draw.io (diagrams.net).
2. Создайте новую схему: **File -> New -> Blank Diagram**.
3. Выставьте формат страницы A4 (Меню **Файл -> Параметры страницы**).
4. Соберите блок-схему по ГОСТ 19.701-90:
 - Блок начала/конца (овал): Начало.
 - Процесс (прямоугольник): Открыть окно входа.
 - Ввод/вывод (параллелограмм): Ввод логина и пароля / выбор "Войти как гость".

- Решение (ромб): Гость?.
- Процесс: Показать список товаров (роль Гость) (ветка Да).
- Решение: Логин/пароль верны? (ветка Нет -> Сообщение об ошибке -> возврат к вводу).
- Процесс: Определить роль (клиент/менеджер/администратор) (ветка Да).
- Процесс: Показать список товаров.
- Процесс: Поиск/фильтр/сортировка.
- Решение: Роль позволяет редактирование?.
- Процесс: CRUD товаров и заказов (для менеджера/администратора).
- Блок начала/конца (овал): Выход.

5. Соедините блоки стрелками по потоку выполнения.

6. Сохраните исходник схемы:

- C:\shoe_store_2026_pu_python\docs\algorithm_gost.drawio

7. Экспортируйте в PDF:

- C:\shoe_store_2026_pu_python\docs\algorithm_gost.pdf

Шаг 13. Подготовьте DOCX со скриншотами корректной работы

Что делаем

Соберите скриншоты основных сценариев.

Команды/код

Создайте файл:

- C:\shoe_store_2026_pu_python\docs\report_screenshots.docx

Добавьте скриншоты:

- окно входа;
- вход как гость;
- вход под менеджером;
- вход под администратором;
- поиск/фильтрация/сортировка;
- добавление/редактирование/удаление товара;
- окно заказов;
- добавление/редактирование/удаление заказа.

Шаг 14. Экспортируйте SQL-скрипт БД и ER-диаграмму

Что делаем

Сохраните итоговую структуру и данные БД.

Команды/код

1. В phpMyAdmin выберите БД shoe2026_pu.
 2. Вкладка **Экспорт** -> формат SQL.
 3. Сохраните как:
 - C:\shoe_store_2026_pu_python\sql\shoe2026_pu.sql
 4. Вкладка **Ещё** -> **Дизайнер** -> экспорт в PDF.
 5. Сохраните как:
 - C:\shoe_store_2026_pu_python\sql\shoe2026_pu_er.pdf
-

Шаг 15. Соберите .exe через PyInstaller

Что делаем

Соберите исполняемый файл приложения.

Команды/код

```
cd C:\shoe_store_2026_pu_python\app
.\venv\Scripts\activate
pyinstaller --noconfirm --windowed --onefile --name ShoeStore2026PUApp --icon ..
\resources\Icon.ico --add-data "..\resources;resources" --collect-all
mysql.connector main.py
```

Результат:

- C:\shoe_store_2026_pu_python\app\dist\ShoeStore2026PUApp.exe
-

Шаг 16. Подготовьте финальный набор файлов и git-коммит

Что делаем

Проверьте комплект итоговых материалов и зафиксируйте локальный коммит.

Команды/код

Проверьте наличие:

- исходный код приложения (структура папок, не архив);
- dist\ShoeStore2026PUApp.exe;
- sql\shoe2026_pu.sql;
- sql\shoe2026_pu_er.pdf;
- docs\algorithm_gost.pdf;
- docs\report_screenshots.docx.

Зафиксируйте итог в локальном git-репозитории:

```
cd C:\shoe_store_2026_pu_python
git add .
```

```
git status  
git commit -m "Финальная версия проекта (Python, Вариант 1 2026)"
```